

PDFTrusted — Complete 360° Technical Intelligence Report

Document version: 2.0

Audit date: 20 May 2026

Project: PDFTrusted (pdftrusted + backend-service)

Stack: Next.js 15 · Vercel · Supabase · Railway Redis · Cloudflare R2 · Railway Workers (7 pools)

Auditor role: Full-Stack · Security · DevOps · Database · OCR/PDF Infrastructure

Related docs: PRODUCTION-LAUNCH.md , OCR_COLOR_PRESERVATION.md , ENTERPRISE_PLATFORM_AUDIT.md , OSS_PDF_PLATFORM_BLUEPRINT.md

Table of Contents

- Executive Summary
 - Full Architecture Overview
 - Tool-by-Tool Breakdown
 - Browser vs Cloud Processing Matrix
 - Libraries & Engines Report
 - Backend Analysis
 - Frontend Analysis
 - Database & Storage Analysis
 - OCR & AI Systems
 - Security Audit
 - Performance Audit
 - UX/UI Quality Audit
 - Infrastructure & Deployment Review
 - Missing Features
 - Scalability Recommendations
 - Technical Debt Report
 - Production Readiness Score
 - Critical Fix Priority List
 - Long-Term Improvement Roadmap
 - Appendix — Cloud Job Flow
 - हिंदी सारांश
-

1. Executive Summary

PDFTrusted is a **hybrid PDF SaaS**: most structural tools run **entirely in the browser** (privacy-first); **13 tools** offload to **Railway Python workers** via **Redis queues**, **Cloudflare R2**, and **Supabase** for auth, quotas, and job state.

Dimension	Assessment
Architecture	Mature split: Vercel orchestration + pool-isolated workers
Cloud OCR / compress / office	Production-grade OSS stack (OCRmyPDF, Ghostscript, LibreOffice)
Browser tools	Strong for merge/split/rotate; weak for layout conversions
AI / LLM	None in production — translate/chat are extract-only or coming soon
Security	Good upload key scoping + worker HMAC; gaps on browser crypto & MIME depth
Deploy readiness	76 / 100 — needs Railway redeploy (59151f5+), smoke 10/10, Vercel sync

Strategic position: Real cloud document engine for OCR, compress, office, and security; browser tier is honest but must not be marketed as Acrobat-class conversion.

Catalog size: 42 tool slugs in constants/tools.js ; **36 live, 6 coming soon** (constants/toolStatus.js).

2. Full Architecture Overview

2.1 Request flow (cloud)

```
User browser
  → Next.js API (Vercel)
    → Presign upload → Cloudflare R2 (enhanced/input/{userId}/)
    → POST /api/enhanced/jobs → Supabase processing_jobs + Redis LPUSH
  → Railway worker (pool: ocr|docx|compress|excel|office|security|convert)
    → RPOPLPUSH claim → download R2 → process → upload output
    → POST /api/enhanced/worker/callback (HMAC + secret)
  → User polls GET /api/enhanced/jobs/{id}
  → Download GET /api/enhanced/jobs/{id}/download (same-origin proxy)
```

2.2 Runtime layers

Layer	Technology	Role
Hosting	Vercel	Next 15 App Router, API routes, cron
Client router	Wouter in NextAppShell	SPA tool UX (src/App.tsx)
Auth	Supabase SSR	OAuth, session cookies
Queue	Redis (Railway TCP)	enhanced:queue:{pool}

Layer	Technology	Role
Storage	R2 (S3 API)	Input/output object keys
Workers	Docker <code>Dockerfile.worker</code>	<code>python -m app.queue.consumer {pool}</code>
Observability	<code>job_trace_events</code> , Sentry	Pipeline stages

2.3 Dual-build note (technical debt)

Production uses **Next** (`next build`). **Vite** remains in `package.json` (`dev:vite` , `vite-plugin-pwa`). PWA uses `public/sw.js` + `PwaUpdatePrompt` . Consolidate to one build path long-term.

2.4 Authoritative config files

Concern	File
Tool tiers (browser/hybrid/cloud)	<code>src/lib/processing/toolProfiles.ts</code>
Engine stacks (OSS map)	<code>src/lib/processing/engineMatrix.ts</code>
Tool → worker pool	<code>server/enhanced/config.ts</code>
Worker routing	<code>backend-service/app/job_runner.py</code>
Tool catalog	<code>constants/tools.js</code>

3. Tool-by-Tool Breakdown

3.1 Edit & compress

Tool	Slug	Route	Processing	Libraries / engine
Compress PDF	<code>compress-pdf</code>	<code>/compress-pdf</code>	Hybrid	Browser: pdf-lib; Cloud: Ghostscript + qpdf
PDF Editor	<code>pdf-editor</code>	<code>/pdf-editor</code>	Browser	pdf-lib, pdf.js, Fabric
Translate PDF	<code>translate-pdf</code>	<code>/translate-pdf</code>	Browser	pdf.js text extract — no LLM
OCR PDF	<code>ocr-pdf</code>	<code>/ocr-pdf</code>	Cloud only	OCRmyPDF, Tesseract, OpenCV preprocess, PaddleOCR (ultra)
Redact PDF	<code>redact-pdf</code>	<code>/redact-pdf</code>	Hybrid	Browser: pdf-lib; Cloud: PyMuPDF redaction
Repair PDF	<code>repair-pdf</code>	<code>/repair-pdf</code>	Browser	pdf-lib → MuPDF WASM fallback
Fill PDF	<code>fill-pdf</code>	—	Coming soon	—

Tool	Slug	Route	Processing	Libraries / engine
Compress Images	compress-images	→ /compress-pdf	Coming soon	Redirect
Enhance Image	enhance-image	—	Coming soon	—
AI Scanner	ai-scanner	/tools/ai-scanner	Browser	OpenCV.js, pdf-lib

3.2 Split & merge

Tool	Processing	Libraries
merge-pdf	Browser + Web Worker	pdf-lib, pdfWorkerPool
split-pdf	Browser	pdf-lib
rotate-pdf	Browser	pdf-lib
merge-images	Coming soon	—

3.3 Convert from PDF

Tool	Processing	Browser	Cloud
universal-converter	Browser	pdf.js, jspdf, heuristics	—
pdf-to-word	Hybrid	pdf.js → RTF	pdf2docx + LibreOffice
pdf-to-image / jpg / png	Hybrid	pdf-engine	PyMuPDF → ZIP
pdf-to-excel	Hybrid	pdf.js → XLSX	pdfplumber + Camelot
pdf-to-pptx	Cloud only	—	PyMuPDF + python-pptx
pdf-to-html	Browser	pdf.js	—
pdf-to-epub	Browser	fflate + custom	—
chat-pdf	Coming soon	—	—

3.4 Convert to PDF

Tool	Processing	Engine
pdf-maker	Browser	pdf-lib
word-to-pdf, pptx-to-pdf	Cloud	LibreOffice headless
png/jpg-to-pdf	Browser	jspdf
excel-to-pdf	Browser	custom + pdf-lib
epub-to-pdf	Coming soon	Cloud profile disabled

3.5 Student essentials

Tool	Processing	Stack
document-scanner	Browser	Canvas + vision pipeline
photo-resizer	Browser	Canvas
resume-builder	Browser	Local storage + PDF export

3.6 Sign & security

Tool	Processing	Notes
protect-pdf / unlock-pdf	Hybrid	Browser: pdf-lib (limited); Cloud: qpdf AES-256
hard-lock-pdf	Browser	Rasterize/flatten
watermark-pdf, page-numbers	Browser	pdf-lib
sign-pdf	Browser	pdf-lib + signature canvas
remove-watermark	Browser	/magic-eraser — canvas erase
generate-qr-code	Browser	qrcode.react

UI shells: 27 dedicated pages in `src/route-pages/tools/` ; others via `ToolPage.tsx` + `src/tools/toolPipeline/registry.ts` .

4. Browser vs Cloud Processing Matrix

Tier	Approx. count	Worker-backed slugs
Browser only	~25 live	merge, split, rotate, editor, sign, repair, html, epub, student, etc.
Hybrid	10 profiles	compress, pdf-to-word/excel/image*, protect, unlock, redact
Cloud only (active)	4	ocr-pdf, word-to-pdf, pptx-to-pdf, pdf-to-pptx

Why this split?

Pattern	Reason
Browser	Privacy, zero marginal cost, instant feedback, AdSense-friendly
Cloud	OCR, Ghostscript, LibreOffice, qpdf, table extraction
Hybrid	Free browser tier; Premium cloud for fidelity (50 MB / 50 pages)

5. Libraries & Engines Report

5.1 Frontend (`package.json`)

Library	Version	Purpose	Grade
next	15.5.18	App + API	Production
pdf-lib	1.17.1	Structural PDF	Production
pdfjs-dist	5.7.284	Render + extract	Production (heavy bundle)
mupdf	1.27.0	Repair WASM	Production
jspdf	4.2.1	Image → PDF	Production
fabric	7.3.1	PDF editor	Production (heavy)
@techstark/opencv-js	4.12	Scanner	Use with care (size)
xlsx	0.18.5	Browser Excel	Keep updated
@supabase/ssr	0.10.3	Auth	Production
ioredis	5.10.1	Vercel queue client	Production
wouter	3.3.5	Client routes	Production

5.2 Backend (`backend-service`)

Engine	Role	Grade
ocrmypdf + Tesseract	OCR	Industry standard
Ghostscript + qpdf	Compress / security	Production
LibreOffice	Office → PDF	Production
PyMuPDF	Images, redact, inspect	Production
pdf2docx	Word export	Good; layout limits
pdfplumber + Camelot	Excel	Production (Camelot optional build)
OpenCV (Python)	OCR preprocess	Fixed RGB/BGR in <code>59151f5</code>
PaddleOCR	Ultra OCR	Optional <code>ENABLE_PADDLE_OCR=1</code>

Policy: No paid APIs (Adobe, Aspose, OpenAI) in production dependency tree.

6. Backend Analysis

6.1 API routes (`app/api/` — 30 routes)

Group	Routes	Function
Enhanced	presign, jobs, jobs/[id], download, callback, usage, health	Job lifecycle
Auth	auth/[...slug]	Supabase session
R2 / multipart	presign-put, multipart/*, delete-staged	Large uploads
Account	register, forgot/reset password	Email (Resend)
Cron	cron/r2-staging-purge	Purge, DLQ, stuck jobs
Internal	internal/job-trace	Ops (gated)

6.2 Worker pools

Pool	Pipelines	Output
ocr	ocr.py, ocr_preprocess.py, paddle_ocr.py	PDF
docx	docx.py, layout_analyze.py	DOCX
excel	excel.py, statement_parse.py	XLSX
compress	compress.py	PDF
office	office.py	PDF
security	pdf_security.py	PDF
convert	convert.py, pdf_images.py, pdf_to_pptx.py	ZIP / PPTX

6.3 Queue semantics

- Enqueue: Vercel `LPUSH` → `enhanced:queue:{pool}`
- Claim: Worker `RPOPLPUSH`
- Retries: Max 3 → `enhanced:dead:{pool}`
- Orphan recovery: Cron `requeueOrphanedProcessing` (900s)

6.4 Weak points

1. Single Redis — burst fairness limited to IP/user caps
 2. Subprocess-bound workers — scale via Railway replicas
 3. `docling_optional.py` not wired to `job_runner`
 4. Encrypted PDF rejected except security pool — UX must explain
-

7. Frontend Analysis

7.1 Structure

- App Router: `app/[locale]/[...path]/page.tsx` → `NextAppShell` → Wouter `App.tsx`
- i18n: en, hi, zh, ar, es, fr, de
- Contexts: Process, ProcessingMode, Premium, AuthPrompt, WorkspaceHistory, TrustShield, PWA

7.2 Key patterns

- `SinglePdfToolShell` — upload → process → result (before/after previews)
- `usePremiumCloudRun` — cloud job + `idb-keyval` lifecycle
- `MIN_PROCESSING_DURATION_MS = 3000` — artificial minimum UX delay

7.3 Issues

Issue	Severity
PDF Editor bundle on mobile	Medium
Next + Wouter dual routing	Medium (complexity)
Coming soon in some nav surfaces	Medium (trust)
opencv-js on main thread	Medium (jank)

8. Database & Storage Analysis

8.1 Supabase tables

Table	Purpose
profiles, subscriptions	User tier
user_usage	Daily enhanced/browser counts
user_usage_monthly, user_usage_totals	Rollups
processing_jobs	Cloud job state
job_trace_events	Pipeline audit
login_events	Auth audit (IP hash)

RLS: Users read own data; writes via **service role** from Vercel API.

8.2 R2 lifecycle

```
enhanced/input/{userId}/{jobId}-file      → worker reads
enhanced/output/{userId}/{jobId}.ext      → download via API proxy → cron purge
```


Worker temp: `TMP_ROOT/{jobId}/` (Python cleanup).
DB stores: metadata only, not file bytes.

9. OCR & AI Systems

9.1 OCR (cloud only)

Stage	Component
Preprocess (optional)	OpenCV LAB on L channel; RGB→BGR before JPEG
Primary	OCRmyPDF + Tesseract
Color	<code>LeaveColorUnchanged</code> ; <code>*_preserve</code> modes with <code>--optimize 0</code>
Modes	fast, balanced, accurate, clean_scan, ultra (PaddleOCR)
Frontend options	<code>ocrPreserveColors</code> , <code>ocrPreprocess</code> , <code>ocrMode</code> , etc.

Deploy: Railway `pdftrusted-worker-ocr @ backend-service commit 59151f5+` .

9.2 AI / LLM

Feature	Status
chat-pdf	Coming soon — no RAG/LLM
translate-pdf	Text extract only
ai-scanner	OpenCV vision, not generative AI

No OpenAI/Anthropic/Gemini in production dependencies.

10. Security Audit

Area	Finding	Severity
Upload key prefix	<code>enhanced/input/{userId}/</code> enforced	Low
Output key prefix	Callback validates output path	Low
Worker auth	Secret + HMAC signature	Low
Cron	CRON_SECRET Bearer in prod	Low
Internal routes	ALLOW_INTERNAL_OPS + localhost	Low if false in prod
debug/agent-log	404 unless QA mode	Low
Browser protect/unlock	pdf-lib ≠ Acrobat encryption	Medium
Presign MIME	Size/filename; magic bytes on worker	Medium

Area	Finding	Severity
Service role key	High impact if leaked	High (standard)
Rate limits	2 enhanced/day, IP caps, browser limits	Low

11. Performance Audit

Bottleneck	Mitigation
Large JS (pdf.js, fabric, opencv)	Route-level code splitting
3s min processing delay	Make optional per tool
OCR worker CPU	Scale ocr replicas
Download via Vercel proxy	Acceptable for CORS; monitor egress
Trace writes to Supabase	Sample at scale

12. UX/UI Quality Audit

Strengths: Before/after cloud previews; processing badges; Trust Shield; 7 locales; PWA; AdSense + consent gating.

Gaps: Show which path (browser vs cloud) on result; hide coming-soon from sitemap/nav; simplify mobile PDF Editor; OCR preserve/preprocess labels (fixed in `OcrPdf.tsx`).

13. Infrastructure & Deployment Review

Service	Role
Vercel	Frontend + API
Railway	7 workers + Redis
Cloudflare R2	Storage
Supabase	Auth + Postgres
Resend	Account email
Sentry	Client errors

Checklist: `docs/PRODUCTION-LAUNCH.md`

Required: Redeploy all Railway workers @ `59151f5+` ; Vercel prod; R2 PUT CORS; smoke 10/10.

14. Missing Features

Feature	UI	Backend
fill-pdf	Coming soon	—
chat-pdf	Coming soon	No LLM
epub-to-pdf	Coming soon	Cloud disabled
compress/merge images, enhance-image	Coming soon	—
Auto-translate	Future copy	Not implemented
Docling layout	—	Code exists, unwired

15. Scalability Recommendations

1. Per-pool autoscaling from queue depth
2. Redis monitoring at ~10k jobs/day
3. Trace event archival / sampling
4. CDN for pdf.worker + WASM
5. Priority queue for paid tier
6. Regional R2 for enterprise

16. Technical Debt Report

Item	Priority
Vite + Next dual toolchain	Medium
Wouter inside Next	Medium
Browser vs qpdf encryption mismatch	High (document in UI)
Stale ENTERPRISE_PLATFORM_AUDIT (Excel/PPT now live)	Low
docling unwired	Low
xlsx in browser for sensitive docs	Medium
Coming-soon in marketing	Medium

17. Production Readiness Score

Area	Score (/10)
Cloud workers	8.0

Area	Score (/10)
API + auth + quotas	8.5
Browser core tools	7.5
Browser conversions honesty	6.0
Security	7.5
Observability	8.0
Deploy docs + cron	8.0
Smoke / tests	7.0
UX / trust	7.5

Overall: 76 / 100

Deployable with known limits; enterprise SLA requires green smoke + worker redeploy.

18. Critical Fix Priority List

P	Action
P0	Redeploy all Railway workers to <code>backend-service@59151f5+</code>
P0	Internal cloud smoke → 10/10
P1	<code>vercel --prod</code> with latest OCR UI
P1	<code>NEXT_PUBLIC_ALLOW_INTERNAL_OPS=false</code> in production
P1	R2 PUT CORS for production origins
P2	Marketing aligns with browser vs cloud matrix
P2	Coming-soon excluded from sitemap/nav
P3	<code>npm run bundle:report</code> for editor/scanner

19. Long-Term Improvement Roadmap

Phase A (0–4 weeks): Smoke + alerts; public processing matrix; document browser encryption limits.

Phase B (1–3 months): Docling or better pdf-to-word; EPUB cloud worker; Next-only build.

Phase C (3–6 months): Paid priority queue; optional 1-page browser OCR teaser; LLM chat only with explicit privacy model.

Phase D (enterprise): Retention policies, CMK, regional buckets, SOC2-oriented ops.

Appendix — Cloud Job Flow

1. Sign in → POST /api/enhanced/presign → PUT to R2
2. POST /api/enhanced/jobs → Supabase row + Redis enqueue
3. Worker claims job → download → process_job → upload → callback
4. Client polls GET /api/enhanced/jobs/{id}
5. Download GET /api/enhanced/jobs/{id}/download (same-origin)
6. Cron purges stale R2 + fails stuck processing jobs

Cloud tool → pool map

Tool slug	Worker pool
ocr-pdf	ocr
pdf-to-word	docx
pdf-to-excel	excel
compress-pdf	compress
word-to-pdf, pptx-to-pdf	office
protect-pdf, unlock-pdf, redact-pdf	security
pdf-to-image, pdf-to-png, pdf-to-jpg, pdf-to-pptx	convert

हिंदी सारांश

- कुल 42 tools कैटलॉग में; 36 live, 6 जल्द आएंगे।
- 13 tools Railway cloud workers पर चलते हैं।
- AI/LLM production में नहीं — translate सिर्फ text निकालता है।
- Architecture: Vercel + Redis + R2 + Supabase + 7 workers — solid।
- अभी जरूरी: Railway redeploy (59151f5+), smoke 10/10, Vercel prod।
- Production score 76% — launch योग्य; browser conversion को Adobe-level advertise न करें।
- OCR रंग bug fix: docs/OCR_COLOR_PRESERVATION.md।

Generated from codebase scan — 20 May 2026. Re-run audit after major worker or tool changes.